

+ 26

Mock Any Function in C++ Without Changing a Single Line of Code

ILYA ERMAKOV &
BRIAN RUDO-HUTT &
RUIFENG ZHANG



20
26



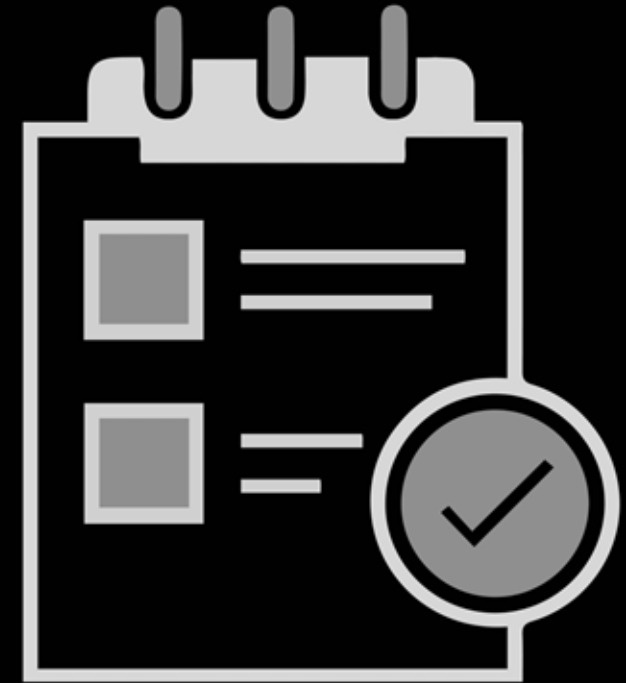
BonoboMock

- Open Source
- Uses monkey patching
- Trivially mocks all kinds of C++ functions
 - Free functions
 - Static functions
 - public/protected/private virtual/non-virtual member methods
 - Local objects
- Creates gMock mocks
 - Set behavior using familiar gMock syntax



Agenda

- The Challenge
- Introducing BonoboMock
- How It Works
- Current State & Future Direction



PART 1

The Challenge

Unit Testing is Needlessly Hard in C++

Bloomberg

Engineering

Function Mocking is Hard in C++

Many functions
**cannot be
overridden** through
virtual dispatch

Dependency
injection may
require **refactoring
production code**

Other methods don't
scale

A Simple, Untestable Function

```
double get_account_balance(int account_id) {  
    auto record = DB::lookup(account_id);           // real database  
    Audit::log("balance_check", account_id);        // real audit  
    return record.balance;  
}
```

Options for Testing

1. Refactor for dependency injection and use gMock
2. Link-seam mocking
3. Containerized environment
4. Bespoke ways

 **All are difficult and some are risky!**

The Refactoring Approach - Interfaces

```
struct IDatabase {  
    virtual ~IDatabase() = default;  
    virtual AccountRecord lookup(int account_id) = 0;  
};  
struct IAuditLogger {  
    virtual ~IAuditLogger() = default;  
    virtual void log(std::string_view event, int account_id) = 0;  
};
```

The Refactoring Approach - Concrete Implementation

```
struct Database : public IDatabase {
    virtual ~Database() = default;
    virtual AccountRecord lookup(int account_id) {
        return DB::lookup(account_id);
    }
};

struct AuditLogger : public IAuditLogger {
    virtual ~AuditLogger() = default;
    virtual void log(std::string_view event, int account_id) {
        Audit::log(event, account_id);
    }
};
```

Rewrite the Function

```
double get_account_balance(int account_id,  
                           IDatabase& db,  
                           IAuditLogger& audit) {  
    audit.log("balance_check", account_id);  
    auto record = db.lookup(account_id);  
    return record.balance;  
}
```

Way Too Hard



22 extra lines for a 3-line body!
Not even counting callers!



"We shouldn't touch working
production code."



Did we introduce a **bug**?



No more static functions.

Link-seam Mocking

```
add_executable(account_balance_test
    account_balance_test.cpp
    db_audit_link_seams.cpp
)

target_link_libraries(account_balance_test PRIVATE
    account_logic
    GTest::gtest_main
)
```

Create TestSeam

```
namespace TestSeam {  
    AccountRecord next_record;  
    std::vector<std::pair<std::string, int>> audit_calls;  
}
```

Create Fakes

```
namespace DB {  
    AccountRecord lookup(int account_id) {  
        return TestSeam::next_record;  
    }  
}  
  
namespace Audit {  
    void log(std::string_view event, int account_id) {  
        TestSeam::audit_calls.push_back(  
            {std::string(event), account_id}  
        );  
    }  
}
```

Not Scalable

Benefits

- Faster / smaller builds
- Production code doesn't change

Downsides

- Need to structure the build system to avoid real dependencies
- Best if can fake all needed symbols
- Build gets complex fast
- Codebase for testing grows
- Tight coupling of fakes and real implementation
- More work to add dynamic behavior

This is Why We Developed BonoboMock

- Why do we accept this limitation?
- We want to:
 - Make it **risk-free** to add tests
 - Make it **trivial** to add tests
- Something is currently happening in the industry...
 - AI needs guardrails

PART 2

Introducing BonoboMock



Bloomberg

Engineering

BonoboMock Design Philosophy

- Make the hard-to-mock easy
 - Mock beyond virtual functions
 - No production code changes
 - Lightweight by design
 - Easy to understand and use
 - Get started in 10 minutes

You Can Mock All Kinds of Functions with 3 Macros

- **BONOBO MOCK**
 - Creates the mock object for all kinds of functions
 - Works together with **BONOBO MOCK FUNCTION**
- **BONOBO MOCK FUNCTION**
 - Works together with **EXPECT_CALL** of Google Test to define how you would like to mock
- **BONOBO MOCK METHOD**
 - A more ergonomic way to create the mock object for member functions

PART 2.1

Mocking Free Functions

Bloomberg

Engineering

Mocking Free Functions

Here is a simple free function:

```
int foo(int& a, int* b, int c) {  
    int result = a + *b + c;  
    a = *b;  
    *b = c;  
    return result;  
}
```

Mocking Free Functions

Mock it with BonoboMock:

```
TEST(Foo, mockFoo) {  
    auto mock = BONOBO MOCK(&foo);  
    int x = 10, y = 11;  
    // int foo(int&, int*, int)  
    EXPECT_CALL(*mock, BONOBO MOCK_FUNCTION(_, _, _))  
        .WillOnce(DoAll(SetArgReferee<0>(x),  
                        SetArgPointee<1>(y),  
                        Return(100)));  
  
    int a = 1, b = 2;  
    EXPECT_EQ(100, foo(a, &b, 3));  
    EXPECT_EQ(10, a);  
    EXPECT_EQ(11, b);  
}
```

Mocking Free Functions

Restore the mocking:

```
TEST(Foo, mockFoo) {  
    // ...  
    // You can restore explicitly  
    // or the mock will be restored by RAII  
    mock->restore();  
    // ...  
}
```

PART 2.2

Mocking Static Member Functions

Continue with the previous example

Bloomberg

Engineering

Let's Revisit the Simple, Untestable Function

```
double get_account_balance(int account_id) {  
    auto record = DB::lookup(account_id);           // real database  
    Audit::log("balance_check", account_id);        // real audit  
    return record.balance;  
}
```

BonoboMock Solution without Code Refactoring

```
struct BalanceMocks {  
    BONOBO MOCK_METHOD(DB, lookup); // Original classes with static members  
    BONOBO MOCK_METHOD(Audit, log);  
};  
  
TEST(AccountTest, ReturnsBalance) {  
    BalanceMocks mock;  
    EXPECT_CALL(mock, lookup(42))  
        .WillOnce(Return(AccountRecord{42, 1500.00}));  
    EXPECT_CALL(mock, log(42));  
    EXPECT_DOUBLE_EQ(1500.00, get_account_balance(42));  
}
```

PART 2.3

Mocking Public Member Functions

Bloomberg

Engineering

Mocking Public Member Functions

A sample class with both public non-virtual and virtual member functions:

```
class SampleClass {  
public:  
    int member_function(bool in);  
    virtual int virtual_function(int in);  
};
```

Mocking Public Member Functions

Define the mock class using **BONOBO MOCK METHOD**:

```
struct SampleClassMock {  
    BONOBO MOCK METHOD (SampleClass, member_function);  
    BONOBO MOCK METHOD (SampleClass, virtual_function);  
};
```

Mocking Public Member Functions

Mock the non-virtual function with BonoboMock (**matching any object, including the local ones**):

```
TEST(Nonvirtual, Mock_nonvirtual) {  
    // Mock int member_function(bool)  
    SampleClassMock mock;  
    EXPECT_CALL(mock, member_function(  ,   ))  
        .Times(1)  
        .WillOnce(Return(100));  
  
    // ...  
}
```

Match any object

Mocking Public Member Functions

Mock the virtual function and set output reference:

```
TEST(Virtual, Mock_virtual) {  
    // int virtual_function(int in)  
    SampleClassMock mock;  
    EXPECT_CALL(mock, virtual_function(_, _))  
        .Times(1)  
        .WillOnce(Return(1000));  
    // ...  
}
```

PART 2.4

Mocking Inaccessible Functions

Protected & Private

Bloomberg

Engineering

Mocking Protected Functions

Need to define a new class to extract protected function address; rest is the same:

```
class childClass : public SampleClassProtected {  
public:  
    using FUNC_TYPE = int(childClass::*)(int*);  
  
    static FUNC_TYPE get_func() {  
        return &childClass::protected_member_function;  
    }  
};
```

Mocking Protected Functions

Cannot create mock class, use **BONOBO MOCK** instead:

```
TEST(Nonvirtual, Mock_protected) {  
    // Mock specific member function directly  
    // Use BONOBO MOCK_VIRTUAL for virtual function in old C++  
    auto mock = BONOBO MOCK(childClass::get_func());  
    EXPECT_CALL(*mock, BONOBO MOCK_FUNCTION(_, _))  
        .Times(1)  
        .WillOnce(Return(100));  
  
    // ...  
}
```

Mocking Private Functions

- Complete support for all kinds of functions
- Can be used for testing legacy code before refactoring
- Use **BONOB0 MOCK PRIVATE *** with **BONOB0 MOCK FUNCTION**

Constructors and Destructors

```
struct DataLookup {  
    ExpensiveExternalResource res;  
  
    DataLookup() : res(initialize_expensive_external_resource()) {}  
  
    std::string get_data();  
};
```

- Constructors may have side effects
 - e.g., members with RAI
- Hard to test the class – can't create an object in isolation
- **BONOB_O_MOCK_DUMMY_CONSTRUCTOR(...)** – dummy the constructor
 - or mock `initialize_expensive_external_resource()`

Comparison

Without BonoboMock:

- Most functions can be mockable with code change
 - May need to touch production code

With BonoboMock:

- All types of functions are mockable without code change
 - Safer
 - Easier to get the buy-in from the team to add tests
- Highly ergonomic
- Previously hard-to-test code paths become much easier

PART 3

How It Works

Replacing a function with a mock & configuring mock behavior

Bloomberg

Engineering

What happens when you create a Mock?

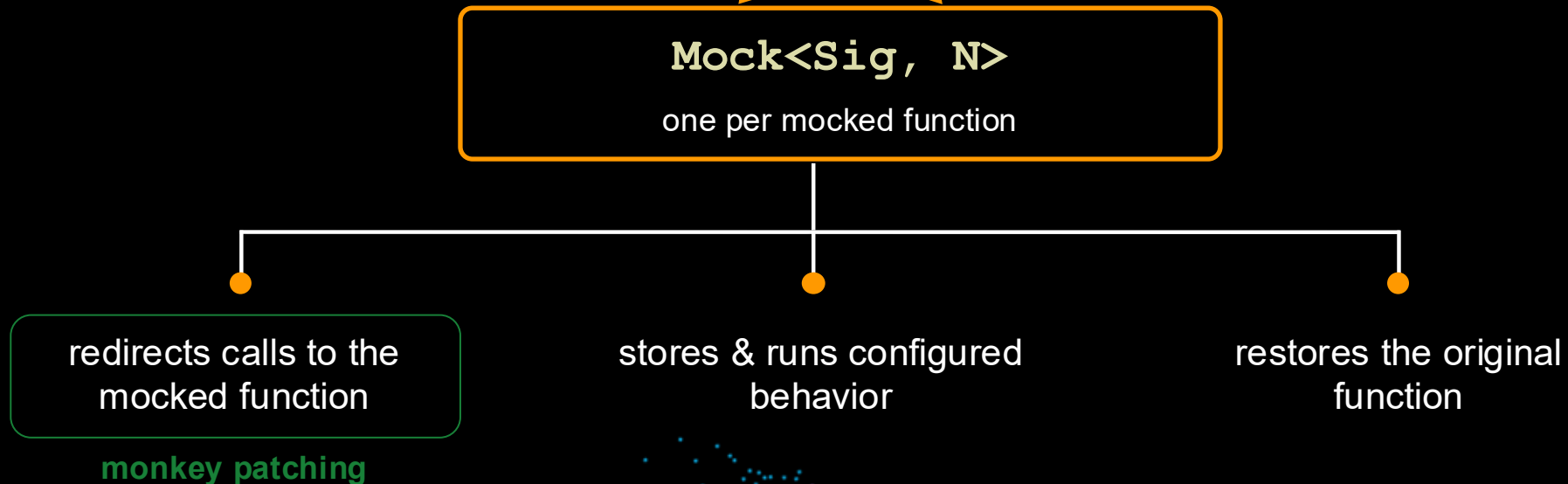
Mock a whole class

```
struct UtilityMock {  
    BONOBO MOCK_METHOD (Utility, compute);  
    // potentially more mocked methods here  
};
```

Mock one function

```
auto mock =  
    BONOBO MOCK (&Utility::compute);
```

one BONOBO MOCK per method



foo() Is Just Bytes at an Address

foo.cpp

```
int foo(int& a, int* b, int c) {  
    int result = a + *b + c;  
    a = *b;  
    *b = c;  
    return result;  
}
```

x86-64 · g++ -O0

→

```
foo(int&, int*, int):  
    push rbp                ; 55  
    mov rbp, rsp            ; 48 89 e5  
    mov QWORD PTR [rbp-0x18], rdi ; 48 89 7d e8  
    mov QWORD PTR [rbp-0x20], rsi ; 48 89 75 e0  
    mov DWORD PTR [rbp-0x24], edx ; 89 55 dc  
    ...  
    pop rbp                 ; 5d  
    ret                     ; c3
```

&foo = 0x401136: 55 48 89 e5 ...

Bloomberg

Engineering

Creating the Mock: What `BONOBO MOCK (&foo)` Does

- 1

Resolve the target address

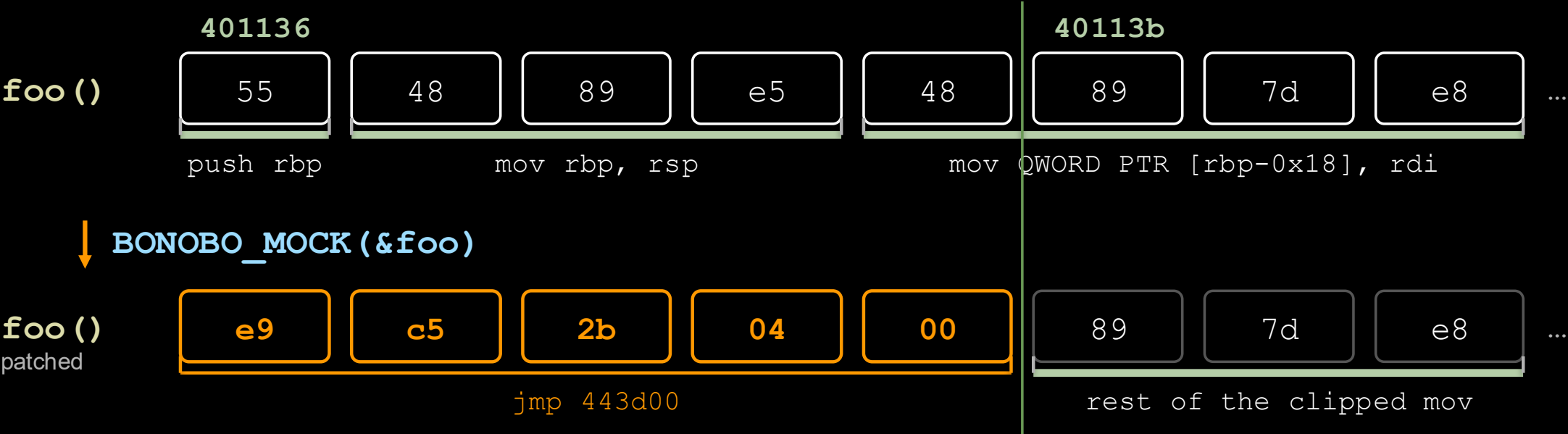
`&foo` → `0x401136`
- 2

Save the original bytes

`55 48 89 e5 48`
- 3

Write a `jmp` at runtime

to `Mock<Sig, N>::mockFunction`



How Google Mock Works

- We can jump into a function we control. Now it has to behave the way the test asked.
- **GMock already solves that. Here is how plain GMock does it.**

```
struct MockUtility : Utility {  
    MOCK_METHOD(int, compute,  
                (int,int));  
};
```

generates

```
gmock_compute(matchers...)  
the method EXPECT_CALL calls
```

```
FunctionMocker d_gmock_compute  
holds the expectation
```

```
EXPECT_CALL(mock, compute(1, 2))
```

expands to

```
mock.gmock_compute(1, 2)
```

stores into

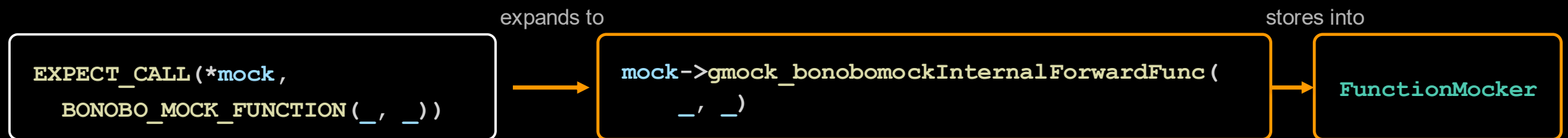
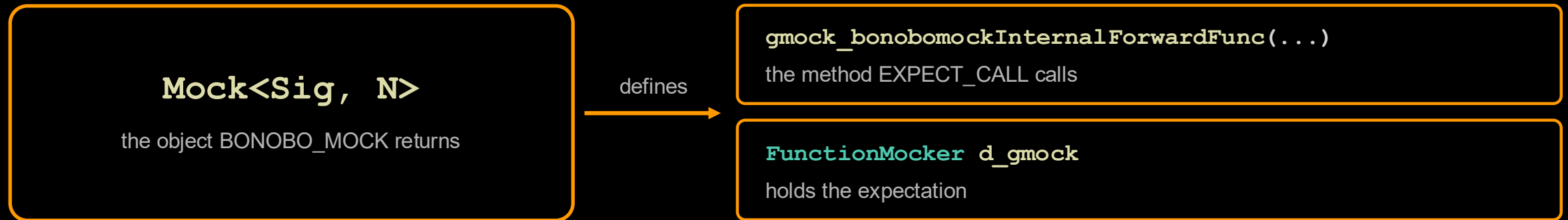
```
FunctionMocker
```

Bloomberg

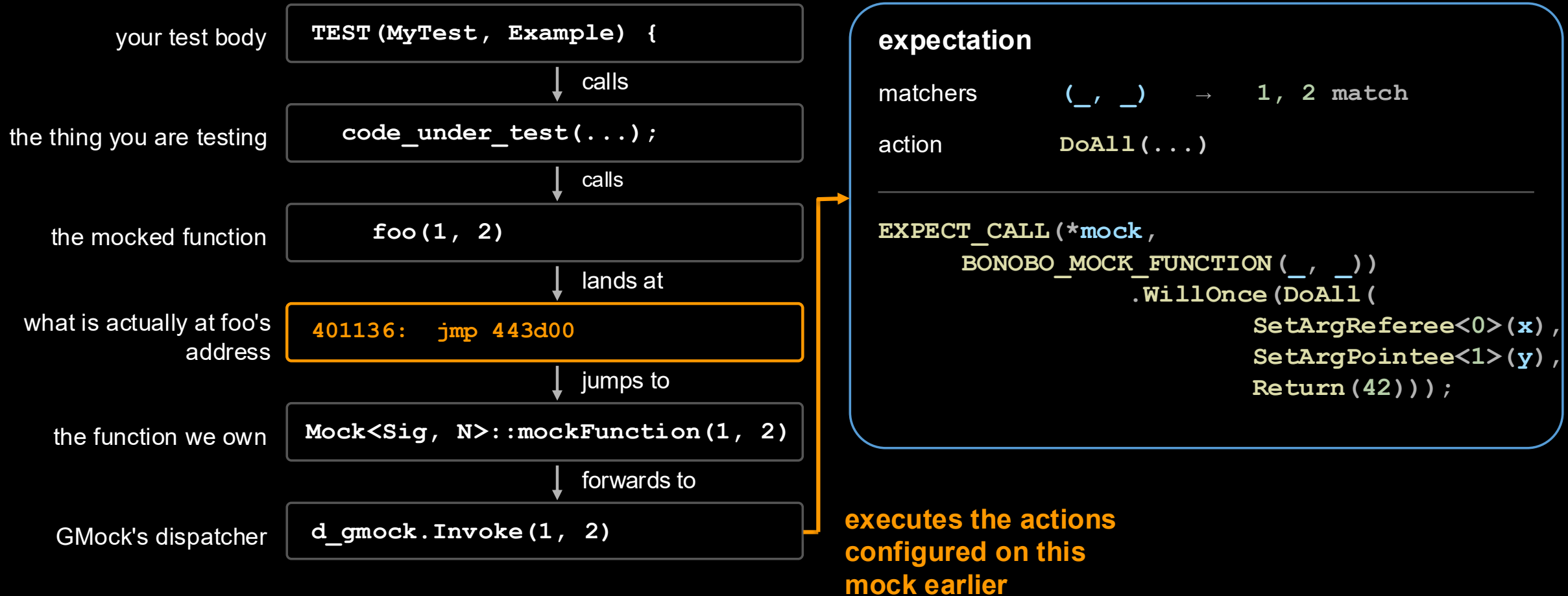
Engineering

Configuring the Mock: What `EXPECT_CALL` Does

- BonoboMock's Mock class defines the same two things.
- That is what allows it to be used with GMock's macros like `EXPECT_CALL`.



Calling the Mock: What Happens Instead



Destroying the Mock: Restore and Verify

The mock object goes out of scope. Destroying it does two things.

1. Restores the binary

patched



saved bytes written back

original



2. Verifies your expectations

`~FunctionMocker`

checks every EXPECT_CALL



✓ the test passes

✗ an unmet expectation fails it here

So What Are We Actually Doing?

01 Find where the function lives

02 At runtime, overwrite its first bytes with a jump

That depends on:

CPU architecture	which jump instructions exist, how many bytes they take, endianness
Operating system	how code pages are made writable, how the instruction cache is synced
Compiler ABI	how a function is represented, e.g. a virtual one is not an address
Application bitness	32- and 64-bit need different jump encodings, and reach different distances
Optimization level	whether the function still has an entry point left to patch

Inlining

What could break?

not inlined

```
caller()  
  call foo
```



```
foo()
```

the caller jumps here

The patch is applied, but the caller runs its own copy.

inlined when optimized

```
caller()  
  <foo body>
```

X

```
foo()
```

still emitted, nothing jumps to it

What we do about it

CMake helper

turns off inlining for the test target

Per-function macros

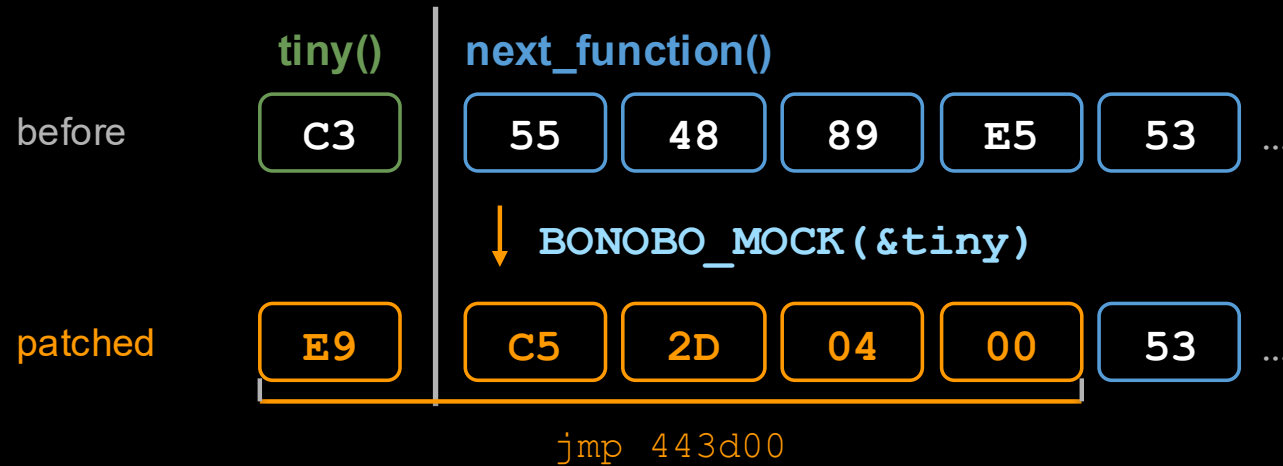
for the ones that still slip through

Unoptimized test

last resort: run that one test at -O0

Functions Too Small to Patch

What could break?



Most likely with trivial functions in optimized builds.

What we do about it

- Use the smallest patch the architecture allows, e.g., 5 bytes on x86-64
- Jump to a nearby trampoline when the jump-to function is far away, so the inline patch stays short

CMake Helper for easier project onboarding

```
find_package(BonoboMockHelper REQUIRED)

add_bonobomock_executable(
    TARGET_NAME      my_test_task
    TEST_FILES       test1.cpp test2.cpp
)

target_link_libraries(my_test_task PRIVATE bonobomock)

include(Catch)
catch_discover_tests(my_test_task)

// OR use GoogleTest, test driver is your choice
// include(GoogleTest)
// gtest_discover_tests(my_test_task)
```

Which Platforms Does BonoboMock Support?

Architectures	Operating systems	Compilers	
x86-64	Linux · RHEL 7+	GCC 12+	full support
x86-64	Linux · RHEL 7+	Clang 18+	full support
x86-64	Windows 10+	MSVC 2022 (v143)+	except lambdas

...plus a few smaller platforms beyond these.

PART 5

Current State & Future Direction

Bloomberg

Engineering

In Use Today

Around a thousand Bloomberg projects

“

It has allowed us to test components without having to rewrite the whole thing.

“

With BonoboMock there is no reason to not have tests.

Bloomberg

Engineering

Philosophy

- BonoboMock is intended to decouple design decisions from testing
- Testing framework no longer enforces a design pattern
 - More responsibility on you to write the code you want to maintain
- Just because you can, doesn't mean you should
 - e.g., mocking private functions



Where We Are Going



Platforms

More platforms (ARM?)



Optimization

Optimizing the linker for
smaller, faster tests?



Future-proofing

Compiler support for
mocking?

Bloomberg

Engineering

Thank you!

And thank you to the other contributors to BonoboMock:

- Jason Carucci
- Brian Maher
- Isaac Gersmann
- Haowen Liu
- Saphal Patro
- Yi Liu
- Dmytro Bagaiev
- Thomas Prideaux-Ghee
- Valentyn Yukhymenko
- Polina Vyshnevskya
- Ryan McKinney
- *and more!*

Bloomberg

Engineering

Key Takeaways: 3 Macros

- **BONOBO MOCK**
- **BONOBO MOCK_METHOD**
- **BONOBO MOCK_FUNCTION**

<https://github.com/bloomberg/bonobomock>



Questions?

Interested in C++ @ Bloomberg?

We are hiring: bloomberg.com/engineering

Engineering

Bloomberg

Career Opportunities



Learn more about our work



TechAtBloomberg.com